Lesson 16 — Introduction to Robot Perception

Physical AI Curriculum | Module 4: Robot Perception | Accelerated lesson
Status: Completed — Python experiment passed; assessment answers B and A correct.
Scope: Introductory sensing, measurement noise and estimation. This lesson does not yet implement an Isaac Sim sensor.

1. Learning objectives

After this lesson, you should be able to:

- Describe the role of perception in the sensing → estimation → action loop.
- Distinguish basic RGB, depth, LiDAR, IMU and encoder measurements.
- Interpret a simple noisy sensor measurement model.
- Explain when averaging improves an estimate and when it cannot correct bias.
- Implement a reproducible noisy distance sensor experiment in Python.

2. Why perception matters in Physical AI

In previous modules, the target for a robot arm was supplied directly to its inverse-kinematics controller. A robot acting autonomously must often *estimate* object position from sensors instead. The practical loop is:

Sensor measurements → state or object estimate → planning / IK → control and action

An image does not directly provide a reliable robot-frame 3D target; camera calibration, depth, and coordinate transforms are needed. Those topics follow in later lessons.

3. Overview of common robot sensors

Sensor	Primary measurement	Example use
RGB camera	Image intensity and color	Detection, recognition
Depth camera	Range/depth to visible surfaces	3D object positioning
LiDAR	Distances to surrounding surfaces	Mapping and obstacle avoidance
IMU	Angular rates and specific force	Orientation/motion estimation
Joint encoder	Joint angle or displacement	Joint-state feedback

Sensors have limitations: random noise, systematic bias, latency, occlusion, limited field of view, and calibration errors.

4. Measurement model and essential formulas

A general measurement model is:

[z = h(x) + v]

where **x** is the physical state, **h(x)** maps the state to a sensor measurement, **v** is measurement noise, and **z** is the observed measurement.

For a simple range sensor:

[z_i = d + v_i]

where **d** is the true distance and **v_i** represents noise on reading **i**.

The sample mean of N measurements is:

[z = _{i=1}^{N}z_i]

If errors are independent and have zero mean with variance σ^2, the sample mean has variance σ^2/N, and its standard deviation is $\sigma/\sqrt{N}$. This does **not** mean the average is always closer to truth for every finite sample. Persistent bias and correlated noise are not automatically eliminated.

5. Worked example

Suppose an object is actually **2.0 m** away and a range sensor has random zero-mean noise with standard deviation **0.10 m**. A single reading can be above or below 2.0 m. Averaging 100 independent readings reduces the *theoretical standard deviation* of the mean to:

[= 0.01;]

This is a prediction about sampling uncertainty, not a guarantee of an exact result.

6. Python hands-on experiment

Save as `lesson_16_sensor_noise.py`. Requires NumPy and Matplotlib.

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)
true_distance = 2.0
noise_std = 0.10
num_samples = 100

measurements = true_distance + rng.normal(0, noise_std, num_samples)
running_average = np.cumsum(measurements) / np.arange(1, num_samples + 1)

single_error = abs(measurements[0] - true_distance)
average_error = abs(running_average[-1] - true_distance)

print(f"True distance: {true_distance:.3f} m")
print(f"First reading: {measurements[0]:.3f} m")
print(f"Average reading: {running_average[-1]:.3f} m")
print(f"First-reading error: {single_error:.4f} m")
print(f"Average error: {average_error:.4f} m")

assert len(measurements) == num_samples
assert np.isfinite(running_average).all()
assert average_error < 0.05  # Check for this reproducible seeded example
print("Verification: PASSED")

plt.figure(figsize=(9, 5))
plt.plot(measurements, ".", alpha=0.4, label="Noisy readings")
plt.plot(running_average, label="Running average")
plt.axhline(true_distance, linestyle="--", label="True distance")
plt.xlabel("Measurement number")
plt.ylabel("Distance (m)")
plt.title("Lesson 16: Sensor Noise and Averaging")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig("lesson_16_sensor_noise.png", dpi=160)
plt.show()
```

Run:

```
python lesson_16_sensor_noise.py
```

Observed result (student-reported): Verification **PASSED**; running-average visualization generated. Specific measured errors were not provided and should not be fabricated.

The random seed ensures reproducible values for the chosen software environment. The assertion is a check for this sample, not a general guarantee that every 100-reading average has error below 0.05 m.

7. Robotics connection

In the existing two-link inverse-kinematics visualizer, target coordinates were entered manually. The perception-enabled extension would estimate the target location from sensor data, transform it to the robot's coordinate frame, assess confidence, then pass it to the IK controller. This introductory experiment demonstrates *sensor uncertainty*, not a complete perception-to-control system.

8. Assessment and answer key

A. Practical: Run sensor simulation and inspect running average.
Result: Passed (student-reported).

B. In the model z = h(x) + v, what is v?
Correct: **B — Measurement noise.** Student answer: **B (correct)**.

C. Why can averaging improve an estimate?
Correct: **A — It can reduce independent, zero-mean random noise.** Student answer: **A (correct)**.

Overall assessment: 3/3 passed; Lesson 16 complete.

9. Key takeaways

1. Perception converts imperfect measurements into information for robot decisions.
2. Sensor readings are not ground truth; uncertainty should be modeled explicitly.
3. Averaging can reduce independent, zero-mean noise but not necessarily systematic bias.

4. Connecting a camera to IK will require camera geometry, depth and frame transforms.

Next: Lesson 17 — Camera Geometry, Pixels and Depth.
Documentation workflow: PDF + Markdown at every completed lesson; module formula summary and module assessment before advancing modules.